

# FDS 复习—并查集 + 图论

6 月 29 日冲刺

## Part I

# 并查集 (Union-Find / Disjoint Set)

## 1 核心问题

给定  $N$  个元素，支持两种操作：

- **Find(x)**:  $x$  属于哪个集合? (找代表元)
- **Union(x,y)**: 把  $x$  和  $y$  所在的两个集合合并

应用：动态等价关系、Kruskal 最小生成树、连通分量检测。

## 2 数据结构

```
1 int S[MAX];    // S[root] = -size (负数表示集合大小)
2                // S[child] = parent index (正数表示父节点)
```

### 2.1 初始化

```
1 void init(int N) {
2     for (int i = 0; i < N; i++) S[i] = -1; // 每个元素自成集合
3 }
```

### 2.2 Find 一路径压缩

```
1 int find(int x) {
2     if (S[x] < 0) return x;           // x 是根
3     return S[x] = find(S[x]);        // 爹变祖宗! 压缩路径
4 }
```

效果：查找路径上所有节点直接指向根，后续查找几乎  $O(1)$ 。

## 2.3 Union —按大小合并

```
1 void unionSet(int a, int b) {
2     a = find(a); b = find(b);
3     if (a == b) return;
4     if (S[a] > S[b]) { int t = a; a = b; b = t; } // a 更大
5     S[a] += S[b];    // 更新大小 (负得更多)
6     S[b] = a;        // b 挂到 a
7 }
```

思路：总是把小的树挂到大的树下，树高不超过  $\lfloor \log_2 N \rfloor + 1$ 。

## 2.4 复杂度

### 考点

Union-by-Size + 路径压缩：

$$T(M, N) = O(M \cdot \alpha(M, N))$$

$\alpha$  是反 Ackermann 函数， $\alpha \leq 4$  在实际数据范围内。

几乎就是常数时间！

- 仅 Union-by-Size:  $T = O(N + M \log N)$
- 朴素 Union (不压缩、不按大小):  $\Theta(N^2)$  最坏

## 3 四个版本对比

| 优化               | Find    | Union |
|------------------|---------|-------|
| 0. 朴素            | 一路走到根   | 随便挂   |
| 1. Union-by-Size | 朴素 Find | 小挂大   |
| 2. 路径压缩          | 爹变祖宗    | 随便挂   |
| 3. 两者都有          | 路径压缩    | 小挂大   |

版本 3 是标准写法。

### 考点

路径压缩不能和 Union-by-Height 一起用！压缩会改变高度，所以改成 Union-by-Rank (秩 = 估计高度)。

## Part II

# 图论 (Graph)

## 4 基本概念

- $G = (V, E)$ ,  $|V|$  顶点数,  $|E|$  边数
- 无向图:  $(v_i, v_j) = (v_j, v_i)$
- 有向图:  $\langle v_i, v_j \rangle \neq \langle v_j, v_i \rangle$
- DAG: 有向无环图
- 握手定理:  $\sum \text{degree}(v) = 2|E|$

## 5 图的表示

### 5.1 邻接矩阵

- $O(|V|^2)$  空间
- 适合稠密图
- 判断两顶点是否相邻:  $O(1)$

### 5.2 邻接表

- $O(|V| + |E|)$  空间
- 适合稀疏图
- 遍历所有边:  $O(|V| + |E|)$

#### 考点

考试判断: 稠密图用矩阵, 稀疏图用表。

## 6 拓扑排序

目标: 给 DAG 的顶点排一个线性顺序, 满足所有边的方向 (前驱在前)。

1. 计算所有顶点入度
2. 入度 = 0 的进队列

3. 出队  $\rightarrow$  所有邻接入度  $-1 \rightarrow$  减到 0 的进队
4. 重复直到队列空
5. 如果输出的顶点数  $< |V| \rightarrow$  图中有环!

时间复杂度:  $O(|V| + |E|)$

```
1 void topsort(Graph G) {
2     Queue Q;
3     for each V
4         if (Indegree[V] == 0) Enqueue(V, Q);
5     int cnt = 0;
6     while (!IsEmpty(Q)) {
7         V = Dequeue(Q);
8         TopNum[V] = ++cnt;
9         for each W adjacent to V
10            if (--Indegree[W] == 0) Enqueue(W, Q);
11    }
12    if (cnt < NumVertex) Error("Graph has a cycle");
13 }
```

## 7 最短路径

### 7.1 无权最短路径—BFS

BFS 就是无权图的最短路径算法。 $O(|V| + |E|)$ 。

### 7.2 Dijkstra 一带权最短路径

思想：贪心。每次选 Dist 最小的未处理顶点，松弛所有邻接点。

条件：不能有负权边！

```
1 void dijkstra(int graph[][MAX], int S, int N) {
2     int known[MAX] = {0};
3     int dist[MAX];
4     for (int i = 0; i < N; i++) dist[i] = INF;
5     dist[S] = 0;
6     for (;;) {
7         // 找未处理中距离最小的
8         int V = -1, minD = INF;
9         for (int i = 0; i < N; i++)
10            if (!known[i] && dist[i] < minD)
```

```

11         { minD = dist[i]; V = i; }
12     if (V == -1) break;
13     known[V] = 1;
14     // 松弛所有邻接点
15     for (int W = 0; W < N; W++)
16         if (graph[V][W] && !known[W])
17             if (dist[V] + graph[V][W] < dist[W])
18                 dist[W] = dist[V] + graph[V][W];
19 }
20 }

```

复杂度：朴素  $O(|V|^2)$ ，用堆优化  $O(|E| \log |V|)$ 。

### 7.3 负权边—Bellman-Ford

每轮松弛所有边，共  $|V| - 1$  轮。 $O(|V| \cdot |E|)$ 。

第  $|V|$  轮如果还能松弛  $\rightarrow$  有负权环！

### 7.4 全源最短路径—Floyd-Warshall

```

1 for (int k = 0; k < N; k++)
2     for (int i = 0; i < N; i++)
3         for (int j = 0; j < N; j++)
4             if (dist[i][k] + dist[k][j] < dist[i][j])
5                 dist[i][j] = dist[i][k] + dist[k][j];

```

$O(|V|^3)$ 。

## 8 最小生成树 (MST)

### 8.1 Prim

思想：类似 Dijkstra，但 Dist 是从当前树到各顶点的最短距离。

复杂度：朴素  $O(|V|^2)$ ，堆优化  $O(|E| \log |V|)$ 。

### 8.2 Kruskal

思想：排序所有边 + Union-Find 判环。

```

1 void kruskal(Graph G) {
2     T = {};
3     sort(edges by weight);
4     while (T has < |V|-1 edges) {

```

```

5      pick min edge (u,v);
6      if (find(u) != find(v)) { // 不同集合 → 不成环
7          add to T;
8          union(u, v);
9      }
10 }
11 }

```

复杂度：  $O(|E| \log |E|) = O(|E| \log |V|)$ 。

### 考点

#### Prim vs Kruskal:

- Prim: 类似 Dijkstra, 适合稠密图
- Kruskal: 排序边 + 并查集, 适合稀疏图

## 9 AOE 网络与关键路径

- $EC[j]$  = 活动  $j$  的最早完成时间 =  $\max_{i \rightarrow j}(EC[i] + C_{ij})$
- $LC[i]$  = 活动  $i$  的最晚完成时间 =  $\min_{i \rightarrow j}(LC[j] - C_{ij})$
- **Slack Time** =  $LC - EC$ , 为 0 的边在**关键路径**上

## 10 网络流—Ford-Fulkerson

核心技巧: 残差图 + 反向边 (撤销机制)。

1. 在残差图中找一条增广路
2. 增加正向边流量, 减少反向边 (等于增加残差)
3. 重复直到不存在增广路

## 11 DFS 应用

### 11.1 双连通分量与关节点

- $Num[v]$  = DFS 编号
- $Low[v]$  = 通过回边能到达的最小  $Num$
- **关节点条件**:  $Low[child] \geq Num[v]$ , 且  $v$  不是根 /  $v$  是根且有  $\geq 2$  个孩子

## 11.2 欧拉回路

- 无向图有欧拉回路  $\iff$  所有顶点度数为偶数
- 无向图有欧拉路径  $\iff$  恰好两个顶点度数为奇数
- 有向图有欧拉回路  $\iff$  每个顶点入度 = 出度